

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 3

Compose Multiplatform Basics

Layouts, UI Components, dan Modifiers

Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun Akademik Genap 2025/2026



**Compose
Multiplatform**

OUTLINE PERTEMUAN

01

Review Minggu Lalu

Coroutines dan Flow

02

Apa itu Compose?

Paradigma UI Deklaratif

03

Composable Functions

Building Blocks UI

04

Basic Layouts

Column, Row, dan Box

05

Modifiers

Styling dan Positioning

06

UI Components

Text, Button, Image, Card, dll

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu membangun UI multiplatform menggunakan Compose

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami paradigma UI deklaratif pada Compose Multiplatform
- ✓ Membuat dan menggunakan Composable functions
- ✓ Menerapkan layout dasar: Column, Row, dan Box
- ✓ Menggunakan Modifier untuk styling dan positioning
- ✓ Mengimplementasikan komponen UI: Text, Button, Image, Card

REVIEW MINGGU LALU

Pertemuan 2: Advanced Kotlin, Coroutines, dan Flow

Null Safety

Elvis operator (?:), Safe call (?.)

Extension Functions

Menambah fungsi ke class yang ada

Coroutines

Asynchronous programming dengan suspend functions

Flow

Reactive streams untuk data yang berubah

💡 **Coroutines dan Flow akan kita gunakan untuk mengambil data dari network dan database!**

BAGIAN 1

APA ITU COMPOSE MULTIPLATFORM?

UI Toolkit Modern untuk Semua Platform

APA ITU COMPOSE MULTIPLATFORM?

Definisi

Compose Multiplatform adalah UI framework deklaratif dari JetBrains yang memungkinkan developer membangun antarmuka pengguna yang SAMA untuk Android, iOS, Desktop, dan Web menggunakan Kotlin.

Perbandingan Paradigma UI:

✗ Imperatif (XML + Code)

```
// Langkah demi langkah
val textView = findViewById(R.id.text)
textView.text = "Hello"
textView.setTextColor(Color.RED)
textView.visibility = View.VISIBLE
```

✓ Deklaratif (Compose)

```
// Deskripsikan APA yang ditampilkan
@Composable
fun Greeting() {
    Text(
        text = "Hello",
        color = Color.Red
    )
}
```

Deklaratif = Anda mendeskripsikan UI, bukan instruksi langkah demi langkah

ANALOGI: MENU RESTORAN

Analogi Pemesanan Makanan

Bayangkan Anda di restoran:

Imperatif: "Ambil piring, taruh nasi, tambah ayam, siram saus, hias dengan daun"

Deklaratif: "Saya mau Nasi Ayam dengan saus spesial"

Imperatif (Cara Lama)

1. Buat TextView
2. Set text "Hello"
3. Set warna merah
4. Set ukuran 24sp
5. Tambahkan ke layout
6. Update manual jika berubah

Deklaratif (Compose)

"Tampilkan teks Hello
dengan warna merah
dan ukuran 24sp"

Otomatis update
jika data berubah!

KEUNTUNGAN COMPOSE MULTIPLATFORM

Single Codebase UI

Satu kode UI untuk Android, iOS, Desktop

Less Code

Lebih sedikit kode dibanding XML tradisional

Intuitive

UI = fungsi Kotlin, mudah dipahami

Preview

Preview UI langsung di IDE

Composable

Komponen dapat digunakan ulang

Modern

Reactive UI dengan state management built-in

"Write once, beautiful everywhere" - Compose Multiplatform

BAGIAN 2

COMPOSABLE FUNCTIONS

Building Blocks dari Compose UI

APA ITU @COMPOSABLE?

@Composable adalah anotasi yang menandai fungsi sebagai UI builder. Fungsi ini mendeskripsikan bagian dari UI dan dapat dipanggil dari fungsi @Composable lainnya.

Contoh Dasar

```
@Composable                                // 1. Anotasi wajib
fun Greeting(name: String) {               // 2. Fungsi dengan parameter
    Text(                                   // 3. Composable built-in
        text = "Hello, $name!",
        fontSize = 24.sp,
        color = Color.Blue
    )
}

// Penggunaan:
@Composable
fun App() {
    Greeting(name = "ITERA")               // Panggil seperti fungsi biasa
}
```

💡 Nama fungsi @Composable menggunakan PascalCase (huruf kapital di awal)

ATURAN COMPOSABLE FUNCTIONS

Harus dianotasi `@Composable`

✓ `@Composable fun MyUI() { }`

X `fun MyUI() { Text(...) }`

Hanya dipanggil dari `@Composable`

✓ `@Composable fun A() { B() }`

X `fun normal() { MyComposable() }`

Tidak return value (Unit)

✓ `@Composable fun Card() { ... }`

X `@Composable fun Card(): View`

Gunakan PascalCase

✓ `fun ProfileCard()`

X `fun profileCard()`

COMPOSABLE DENGAN PARAMETER

Reusable Component

```
// Komponen yang dapat digunakan ulang
@Composable
fun ProfileCard(
    name: String,                // Parameter wajib
    title: String = "Mahasiswa", // Parameter dengan default
    avatarUrl: String? = null,    // Parameter nullable
    onClick: () -> Unit = {}     // Callback lambda
) {
    Card(modifier = Modifier.padding(8.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            // Avatar
            if (avatarUrl != null) {
                AsyncImage(model = avatarUrl, modifier = Modifier.size(48.dp))
            }
            // Info
            Column(modifier = Modifier.padding(start = 12.dp)) {
                Text(text = name, fontWeight = FontWeight.Bold)
                Text(text = title, color = Color.Gray)
            }
        }
    }
}

// Penggunaan dengan berbagai parameter
ProfileCard(name = "Budi")
ProfileCard(name = "Ani", title = "Dosen", onClick = { navigateToProfile() })
```

BAGIAN 3

BASIC LAYOUTS

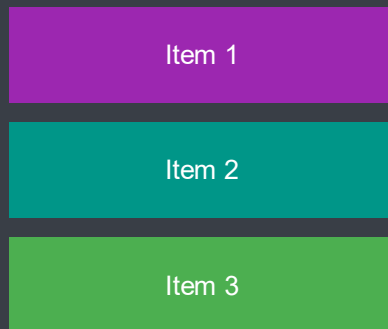
Column, Row, dan Box

Mengatur posisi elemen UI

TIGA LAYOUT UTAMA

Column

Vertikal (atas ke bawah)



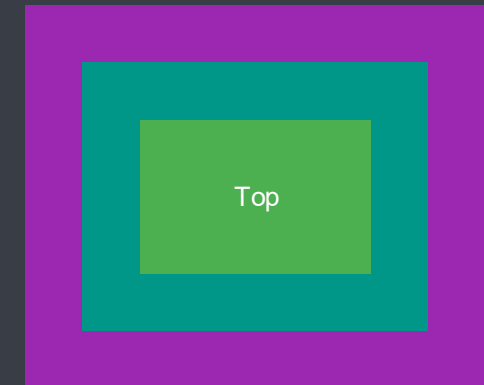
Row

Horizontal (kiri ke kanan)



Box

Stack (tumpuk)



```
Column { } // Susun vertikal
Row { }    // Susun horizontal
Box { }    // Tumpuk (overlap)
```

COLUMN - LAYOUT VERTIKAL

Code

```
@Composable
fun ColumnExample() {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        verticalArrangement =
            Arrangement.spacedBy(8.dp),
        horizontalAlignment =
            Alignment.CenterHorizontally
    ) {
        Text("Header")
        Text("Content 1")
        Text("Content 2")
        Button(onClick = {}) {
            Text("Action")
        }
    }
}
```

Parameter Penting:

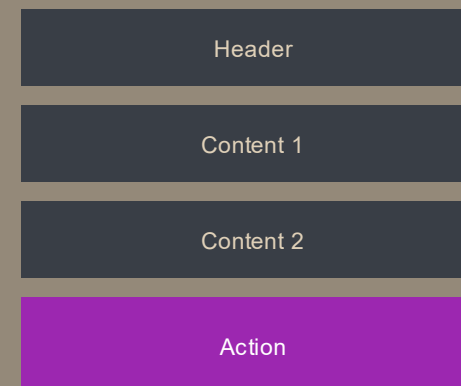
verticalArrangement:

- Top, Bottom, Center
- SpaceBetween, SpaceAround
- spacedBy(8.dp)

horizontalAlignment:

- Start, End, CenterHorizontally

Preview:



ROW - LAYOUT HORIZONTAL

Code

```
@Composable
fun RowExample() {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalArrangement =
            Arrangement.SpaceBetween,
        verticalAlignment =
            Alignment.CenterVertically
    ) {
        Icon(
            Icons.Default.Person,
            contentDescription = null
        )
        Text(
            text = "Username",
            modifier = Modifier.weight(1f)
        )
        Button(onClick = {}) {
            Text("Follow")
        }
    }
}
```

Parameter Penting:

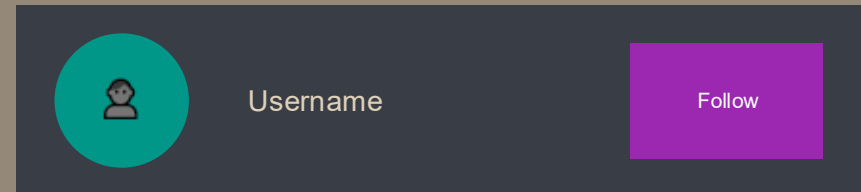
horizontalArrangement:

- Start, End, Center
- SpaceBetween, SpaceAround
- SpaceEvenly

verticalAlignment:

- Top, Bottom, CenterVertically

Preview:



💡 *weight(1f) membuat elemen mengisi sisa ruang*

BOX - LAYOUT STACK

Code

```
@Composable
fun BoxExample() {
    Box(
        modifier = Modifier
            .size(200.dp)
            .background(Color.LightGray),
        contentAlignment = Alignment.Center
    ) {
        // Layer 1 (paling bawah)
        Image(
            painter = painterResource(
                Res.drawable.background
            ),
            contentDescription = null,
            modifier = Modifier.fillMaxSize()
        )

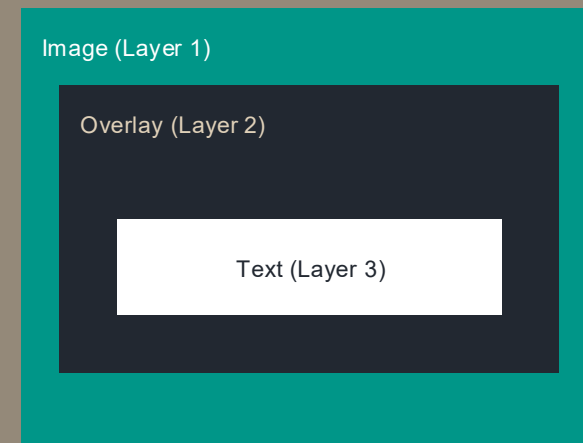
        // Layer 2 (tengah)
        Box(
            modifier = Modifier
                .fillMaxSize()
                .background(Color.Black.copy(alpha = 0.5f))
        )

        // Layer 3 (paling atas)
        Text(
            text = "Overlay Text",
            color = Color.White
        )
    }
}
```

contentAlignment:

- TopStart, TopCenter, TopEnd
- CenterStart, Center, CenterEnd
- BottomStart, BottomCenter, BottomEnd

Preview (Layers):



BAGIAN 4

MODIFIERS

Styling dan Positioning

Mengubah tampilan dan perilaku komponen

APA ITU MODIFIER?

Modifier adalah objek yang memodifikasi tampilan, layout, dan perilaku dari Composable. Modifier di-chain (dirangkai) menggunakan dot notation dan urutan pemanggilan PENTING!

Contoh Modifier Chain

```
Text(  
    text = "Hello Compose!",  
    modifier = Modifier  
        .fillMaxWidth()           // 1. Lebar penuh  
        .padding(16.dp)           // 2. Padding luar  
        .background(Color.Blue)   // 3. Background biru  
        .padding(8.dp)            // 4. Padding dalam  
        .clickable { onClick() }  // 5. Bisa diklik  
)  
  
// URUTAN PENTING!  
// padding -> background -> padding  
// berbeda dengan  
// background -> padding -> padding
```

⚠ *Urutan modifier = urutan eksekusi. Padding sebelum background berbeda dengan padding sesudah background!*

MODIFIER YANG SERING DIGUNAKAN

SIZE

```
fillMaxWidth(), fillMaxHeight(), fillMaxSize()  
size(100.dp), width(50.dp), height(50.dp)  
wrapContentWidth(), wrapContentHeight()
```

PADDING/MARGIN

```
padding(16.dp), padding(horizontal = 8.dp)  
padding(start = 4.dp, top = 8.dp)
```

BACKGROUND

```
background(Color.Red)  
background(Color.Blue, RoundedCornerShape(8.dp))
```

BORDER

```
border(1.dp, Color.Gray)  
border(2.dp, Color.Blue, CircleShape)
```

CLICK

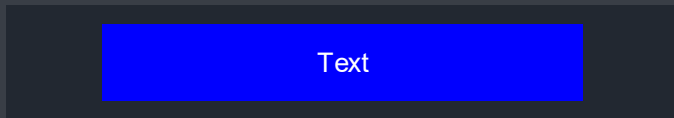
```
clickable { onClick() }  
pointerInput(Unit) { detectTapGestures() }
```

URUTAN MODIFIER PENTING!

Perbedaan urutan padding dan background:

Padding lalu Background

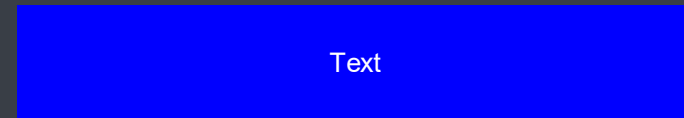
```
Modifier
    .padding(16.dp)
    .background(Color.Blue)
```



Padding di luar, lalu background

Background lalu Padding

```
Modifier
    .background(Color.Blue)
    .padding(16.dp)
```



Background penuh, padding di dalam

💡 Tips Mengingat Urutan:

Bayangkan Anda membungkus kado:

1. Kotak (size) → 2. Isi padding dalam → 3. Kertas kado (background) → 4. Pita luar (border) → 5. Jarak dengan kado lain (padding luar)

BAGIAN 5

UI COMPONENTS

Text, Button, Image, Card, dan lainnya

Komponen UI siap pakai

TEXT - MENAMPILKAN TEKS

Text Component

```
// Basic Text
Text(text = "Hello World!")

// Styled Text
Text(
    text = "Styled Text",
    fontSize = 24.sp,                // Ukuran font
    fontWeight = FontWeight.Bold,    // Ketebalan
    fontStyle = FontStyle.Italic,    // Gaya
    color = Color.Blue,              // Warna
    textAlign = TextAlign.Center,    // Alignment
    maxLines = 2,                    // Maks baris
    overflow = TextOverflow.Ellipsis // Jika overflow
)

// Text dengan Multiple Styles (AnnotatedString)
Text(
    buildAnnotatedString {
        withStyle(style = SpanStyle(color = Color.Red)) {
            append("Red ")
        }
        withStyle(style = SpanStyle(fontWeight = FontWeight.Bold)) {
            append("Bold")
        }
    }
)
```

BUTTON - TOMBOL INTERAKTIF

Button Types

```
// Filled Button (Primary)
Button(
    onClick = { /* action */ },
    colors = ButtonDefaults.buttonColors(
        containerColor = MaterialTheme.colorScheme.primary
    )
) {
    Text("Primary Button")
}

// Outlined Button (Secondary)
OutlinedButton(onClick = { }) {
    Icon(Icons.Default.Add, contentDescription = null)
    Spacer(modifier = Modifier.width(8.dp))
    Text("Add Item")
}

// Text Button (Tertiary)
TextButton(onClick = { }) {
    Text("Cancel")
}

// Floating Action Button
FloatingActionButton(onClick = { }) {
    Icon(Icons.Default.Add, contentDescription = "Add")
}
```


IMAGE - MENAMPILKAN GAMBAR

Image Types

```
// Image dari Resource (lokal)
Image (
    painter = painterResource(Res.drawable.my_image),
    contentDescription = "Deskripsi gambar",
    modifier = Modifier
        .size(100.dp)
        .clip(CircleShape),
    contentScale = ContentScale.Crop
)

// Image dari URL (dengan Coil - perlu library)
AsyncImage (
    model = "https://example.com/image.jpg",
    contentDescription = "Image dari internet",
    modifier = Modifier.fillMaxWidth(),
    contentScale = ContentScale.FillWidth
)

// Icon (Material Icons)
Icon (
    imageVector = Icons.Default.Favorite,
    contentDescription = "Like",
    tint = Color.Red,
    modifier = Modifier.size(24.dp)
)
```

CARD - CONTAINER DENGAN ELEVASI

Card Component

```
@Composable
fun ProductCard(
    product: Product,
    onClick: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(8.dp)
            .clickable { onClick() },
        elevation = CardDefaults
            .cardElevation(4.dp),
        shape = RoundedCornerShape(12.dp)
    ) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            AsyncImage(
                model = product.imageUrl,
                contentDescription = null,
                modifier = Modifier
                    .fillMaxWidth()
                    .height(120.dp),
                contentScale = ContentScale.Crop
            )
            Spacer(Modifier.height(8.dp))
            Text(
                text = product.name,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = product.price,
                color = Color.Gray
            )
        }
    }
}
```

Preview:



Product Name

Rp 150.000

TEXTFIELD - INPUT TEKS

TextField Types

```
// State untuk menyimpan input
var text by remember { mutableStateOf("") }

// Filled TextField (Material 3)
TextField(
    value = text,
    onValueChange = { text = it },
    label = { Text("Username") },
    placeholder = { Text("Masukkan username") },
    leadingIcon = { Icon(Icons.Default.Person, null) },
    trailingIcon = {
        if (text.isNotEmpty()) {
            IconButton(onClick = { text = "" }) {
                Icon(Icons.Default.Clear, "Clear")
            }
        }
    },
    singleLine = true,
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Email,
        imeAction = ImeAction.Next
    )
)

// Outlined TextField
OutlinedTextField(
    value = text,
    onValueChange = { text = it },
    label = { Text("Password") },
    visualTransformation = PasswordVisualTransformation()
)
```

KOMPONEN UI LAINNYA

Checkbox

Pilihan ya/tidak

```
Checkbox (checked = isChecked, onCheckedChange = { })
```

Switch

Toggle on/off

```
Switch (checked = isOn, onCheckedChange = { })
```

RadioButton

Pilihan tunggal

```
RadioButton (selected = isSelected, onClick = { })
```

Slider

Input range nilai

```
Slider (value = sliderValue, onValueChange = { })
```

CircularProgressIndicator

Loading spinner

```
CircularProgressIndicator ()
```

Divider

Garis pemisah

```
Divider (thickness = 1.dp, color = Color.Gray)
```

BAGIAN 6

HANDS-ON PRACTICE

3 Latihan Praktik

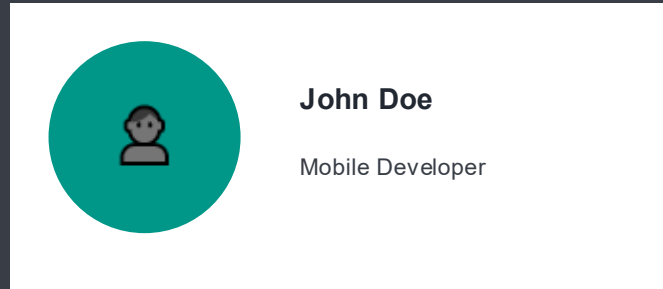
LATIHAN 1: PROFILE CARD

Buat komponen ProfileCard dengan foto, nama, dan bio

Template Code

```
@Composable
fun ProfileCard(
    name: String,
    bio: String,
    avatarUrl: String
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        elevation = CardDefaults.cardElevation(4.dp)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // TODO: Tambahkan Image avatar
            // TODO: Tambahkan Column untuk name dan bio
        }
    }
}
```

Expected Result:



Checklist:

- ☐ Avatar (circular image)
- ☐ Nama (bold)
- ☐ Bio (gray color)
- ☐ Card dengan elevasi
- ☐ Padding yang rapi

LATIHAN 2: LOGIN FORM

Buat form login dengan username, password, dan tombol login

Template Code

```
@Composable
fun LoginForm(onLogin: (String, String) -> Unit) {
    var username by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // TODO: Text judul "Login"
        // TODO: OutlinedTextField untuk username
        // TODO: OutlinedTextField untuk password
        //      (dengan visualTransformation)
        // TODO: Button "Login"
    }
}
```

Expected Result:

The expected result is a login form with a white background. At the top, the word "Login" is displayed in a large, bold, black font. Below the title, there is a light beige rectangular box containing the text "Username" in a dark grey font. Underneath this, there is another light beige rectangular box containing a series of dots, representing a password field. At the bottom of the form, there is a solid teal rectangular button with the word "LOGIN" in white, uppercase letters.

LATIHAN 3: PRODUCT LIST

Buat tampilan list produk dengan gambar, nama, dan harga

Template Code

```
data class Product(  
    val name: String,  
    val price: String,  
    val imageUrl: String  
)  
  
@Composable  
fun ProductItem(product: Product) {  
    // TODO: Buat Card dengan Row  
    // TODO: Image di kiri (square, 80dp)  
    // TODO: Column di kanan (nama + harga)  
}  
  
@Composable  
fun ProductList(products: List<Product>) {  
    Column(  
        modifier = Modifier.fillMaxSize(),  
        verticalArrangement = Arrangement.spacedBy(8.dp)  
    ) {  
        products.forEach { product ->  
            ProductItem(product = product)  
        }  
    }  
}
```

Expected Result:



Product 1

Rp 100.000



Product 2

Rp 250.000



Product 3

Rp 75.000

TROUBLESHOOTING

✗ `@Composable` invocations can only happen from...

Penyebab: Memanggil Composable dari fungsi biasa

✓ Solusi: Pastikan caller juga `@Composable`

✗ `Unresolved reference: dp / sp`

Penyebab: Import belum ditambahkan

✓ Solusi: `import androidx.compose.ui.unit.dp`

✗ `Type mismatch: Color expected`

Penyebab: Menggunakan Color dari package salah

✓ Solusi: Gunakan `androidx.compose.ui.graphics.Color`

✗ `Preview tidak muncul`

Penyebab: Fungsi tidak dianotasi `@Preview`

✓ Solusi: Tambahkan `@Preview` di atas `@Composable`

RANGKUMAN PERTEMUAN 3

Compose Multiplatform

UI framework deklaratif untuk semua platform

@Composable

Fungsi yang mendeskripsikan UI, menggunakan PascalCase

Basic Layouts

Column (vertikal), Row (horizontal), Box (stack)

Modifiers

Chain untuk styling, URUTAN PENTING!

UI Components

Text, Button, Image, Card, TextField, dll

 **Key Takeaway: Compose = Deklaratif, Reusable, dan Konsisten di semua platform!**

TUGAS PRAKTIKUM MINGGU 3

Bobot: 4% | Deadline: Sebelum Pertemuan 4

Deskripsi Tugas:

Buat aplikasi "My Profile App" dengan fitur:

1. Halaman Profile:
 - Header dengan foto profil (circular) dan nama
 - Bio/deskripsi singkat
 - List informasi: Email, Phone, Location
2. Minimal 3 Composable Functions yang reusable:
 - ProfileHeader, InfoItem, ProfileCard (atau custom lainnya)
3. Gunakan minimal: Column, Row, Box, Card, Text, Button, Image/Icon

Format Pengumpulan:

- Push ke GitHub repository yang sama dengan tugas sebelumnya
- README: Screenshot aplikasi di Android/Desktop
- Code harus bisa di-build tanpa error

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
Layout Implementation	25%	Penggunaan Column, Row, Box yang tepat
Reusable Composables	25%	Minimal 3 custom composable functions
UI Components	20%	Penggunaan Text, Button, Image, Card
Modifiers	15%	Styling dan positioning dengan modifier
Code Quality	15%	Clean code, proper naming, dokumentasi

🌟 Bonus (+10%)

Implementasi animasi (AnimatedVisibility)

⚠️ Penalti

Terlambat: -10%/hari, Plagiat: 0

PREVIEW MINGGU DEPAN

Pertemuan 4: State Management dan MVVM

State dalam Compose

remember, mutableStateOf

State Hoisting

Mengangkat state ke parent

MVVM Pattern

Model-View-ViewModel

ViewModel

Menyimpan state yang survive configuration change

Persiapan:

- Selesaikan tugas minggu ini (Profile App)
- Baca dokumentasi: State and Jetpack Compose (developer.android.com)
- Review konsep reactive programming

Terima Kasih

Pertemuan 3: Compose Multiplatform Basics

"The best way to learn Compose is to compose."

- JetBrains

Selamat berlatih membangun UI dengan Compose! 🎨